Подготовка узла для работы с CAN-шиной

Cyphal

Для взаимодействия с узлами будем использовать Cyphal [1]. Основная информация по протоколу описана в гайде (может потребоваться VPN чтобы открыть и прочитать).

В Cyphal есть две основных составляющих:

1. Собственно протокол общения.
2. Конкретные типы с правилами сериализации и десериализации.

Первая часть реализуется с помощью библиотек, на C или C++ для CAN-шины нас интересует libCANard — там реализованы функции для подготовки к отправке и приёму (при этом сам механизм передачи данных на шину и считывания с неё остаётся разработчику).

Вторая часть — dsdl типы. Вообще dsdl это язык, на котором мы в целом можем описать что угодно и добавить свои кастомные типы, а потом автоматически сгенерировать правила сериализации и десериализации. Но в целом есть достаточно большое количество «стандартных типов», поэтому скорее всего нам нет необходимости добавлять свои. Общие типы лежат здесь. `uavcan/node` — всякие полезные технические вещи. `uavcan/si` — единицы СИ.

Архитектурно сеть состоит из улов. Каждый узел имеет свой индекс и может принимать/отправлять данные с каких-либо портов. Некоторые порты зарезервированы для конкретных типов передачи, например 7509 — `Hearbeat`, который каждый узел должен посылать хотя бы раз в секунду, чтобы сообщить о своей активности. Все остальные порты могут принимать/отправлять какие-угодно данные (одного типа).

Если узел «слушает» некоторый порт, тогда ему будет поступать информация со всех узлов, отправляемая в этот порт. Обратно, после публикации данных в некоторый порт их получат все узлы, которые его «слушают».

Помимо режима подписки/публикации есть режим запроса/ответа. Можно отправить запрос в конкретный порт конкретного узла и получить ответ на такой запрос. Отдельно существуют регистры, которые поддерживают прямую запись/чтение чисел.

В общем для того, чтобы можно было работать с некоторым узлом, нужно знать какие типы данных он слушает/отправляет на каких портах, куда можно отправить запрос.

Особенности разработки

Пример работы со всем этим делом на современной версии libCANard есть в репозитории с примерами, а именно `differential_pressure_sensor`. Стоит отметить, что для работы с DSDL типами нужен дополнительный шаг — нужна кодогенерация на соответствующем языке. Для этого используется nunavut. Можно скомпилировать в C или в C++, смотря на чём хотим писать. Правда поддержка C++ экспериментальная.

К настоящему документу прилагается архив с примером проекта, который использует виртуальную CAN-шину на linux. Он написан на основе указанного ранее примера использования libCANard и официального демо для Python. Там уже есть сгенерированные с помощью `nunavut` dsdl типы.

Библиография

[1] P. Kirienko, S. Dixon, и others, «OpenCyphal: an open technology for robust real-time intravehicular distributed computing and communication based on modern networking standards». [Онлайн]. Доступно на: https://opencyphal.org/